LAPORAN PRAKTIKUM

Protokol Komunikasi Telemetri Industri (MQTT)

Oleh:

NAMA : Muhammad Hayya Faza Habibilah

NPM : 25782104

KELAS : TRI – 3D

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

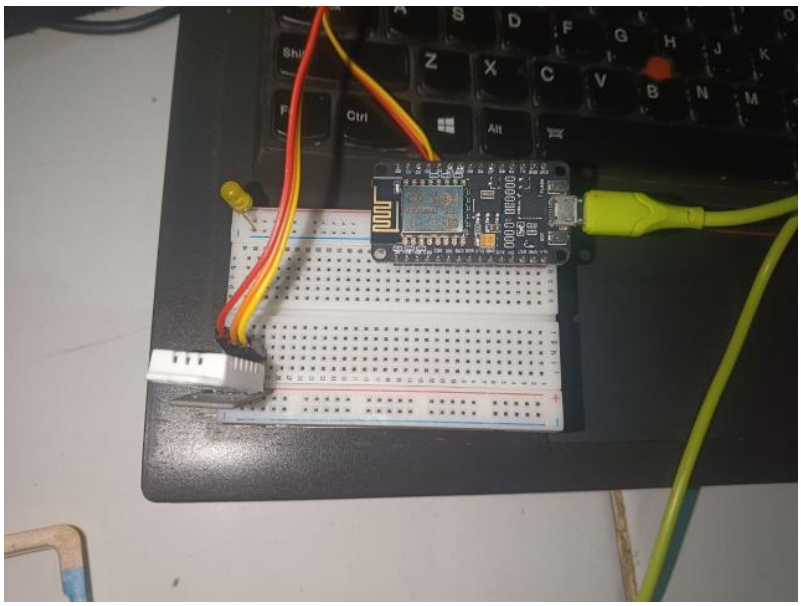
POLITEKNIK NEGERI LAMPUNG

2026/2027

PRAKTIKUM

6.5.1 Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

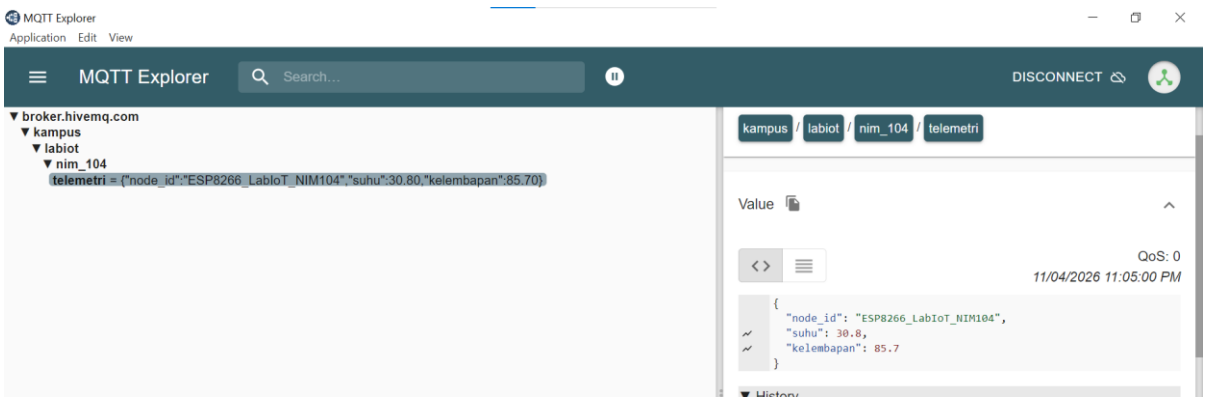
Foto:



Hasil serial:

```
22:59:38.319 -> Menghubungkan ke MQTT Broker... Sukses Terhubung!
22:59:39.865 -> Telemetri Terkirim [kampus/labiot/nim_104/telemetri]: {"node_id":"ESP8266_LabIoT_NIM104","suhu":31.00,"kelembapan":84.90}
22:59:44.834 -> Telemetri Terkirim [kampus/labiot/nim_104/telemetri]: {"node_id":"ESP8266_LabIoT_NIM104","suhu":31.00,"kelembapan":85.10}
22:59:49.820 -> Telemetri Terkirim [kampus/labiot/nim_104/telemetri]: {"node_id":"ESP8266_LabIoT_NIM104","suhu":31.00,"kelembapan":85.00}
22:59:54.830 -> Telemetri Terkirim [kampus/labiot/nim_104/telemetri]: {"node_id":"ESP8266_LabIoT_NIM104","suhu":31.00,"kelembapan":85.10}
22:59:59.834 -> Telemetri Terkirim [kampus/labiot/nim_104/telemetri]: {"node_id":"ESP8266_LabIoT_NIM104","suhu":31.00,"kelembapan":85.20}
23:00:04.844 -> Telemetri Terkirim [kampus/labiot/nim_104/telemetri]: {"node_id":"ESP8266_LabIoT_NIM104","suhu":30.90,"kelembapan":85.20}
23:00:09.826 -> Telemetri Terkirim [kampus/labiot/nim_104/telemetri]: {"node_id":"ESP8266_LabIoT_NIM104","suhu":30.90,"kelembapan":85.40}
23:00:14.858 -> Telemetri Terkirim [kampus/labiot/nim_104/telemetri]: {"node_id":"ESP8266_LabIoT_NIM104","suhu":30.90,"kelembapan":85.40}
23:00:19.825 -> Telemetri Terkirim [kampus/labiot/nim_104/telemetri]: {"node_id":"ESP8266_LabIoT_NIM104","suhu":30.90,"kelembapan":85.50}
```

Hasil MQTT ekspoler:



Hasil pengamatan:

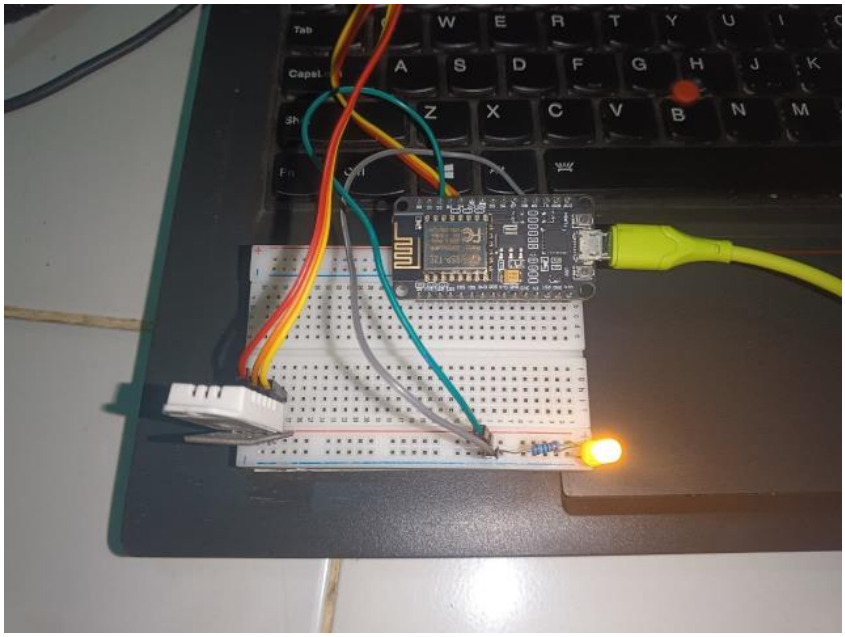
Node berhsil terhubung dengan WI-FI dan mulai menghubungkan dengan MQTT broker dan setelah terhubung Node mengirim Telemetri ke topik “kampus/labiot/nim_104/telemetri

” dan di terima broker dalam bentuk Json.

6.5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Subscriber*. Mikrokontroler mendaftarkan minatnya pada topik perintah (kampus/labiot/nim_anda/perintah). Saat pengguna luar (melalui aplikasi penguji MQTT) memublikasikan perintah berformat JSON, fungsi *callback* NodeMCU akan membedah payload dan menyalakan/mematikan lampu LED indikator secara seketika.

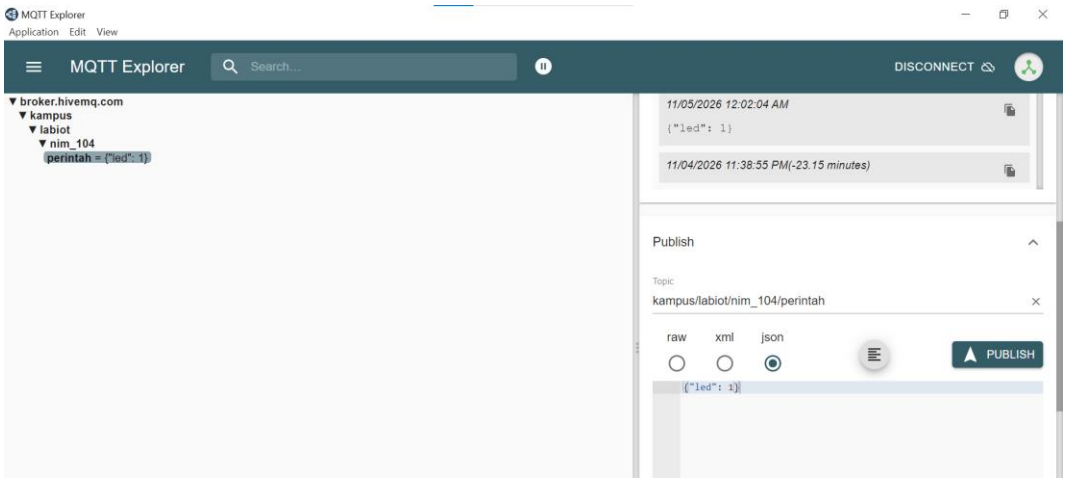
Foto:



Isi serial:

```
......
Wi-Fi Terkoneksi!
Mencoba koneksi MQTT... Terhubung!
Subskripsi Topik: kampus/labiot/nim_104/perintah
=======================================
Pesan Baru Masuk pada Topik: kampus/labiot/nim_104/perintah
Payload Diterima: {"led": 1}
--> Aksi: Status LED diubah menjadi MENYALA (ON)
=======================================
```

Isi MQTT Ekspoler:



Analisis Hasil:

NoteMCU berhasil terhubung ke broker MQTT dan mendaftarkan sebagai subscribe ke topik “kampus/labiot/nim_104/perintah”, dan saat kita beri payload JSON {"led": 1}, nodeMCU akan menerima dan menghidupkan LED

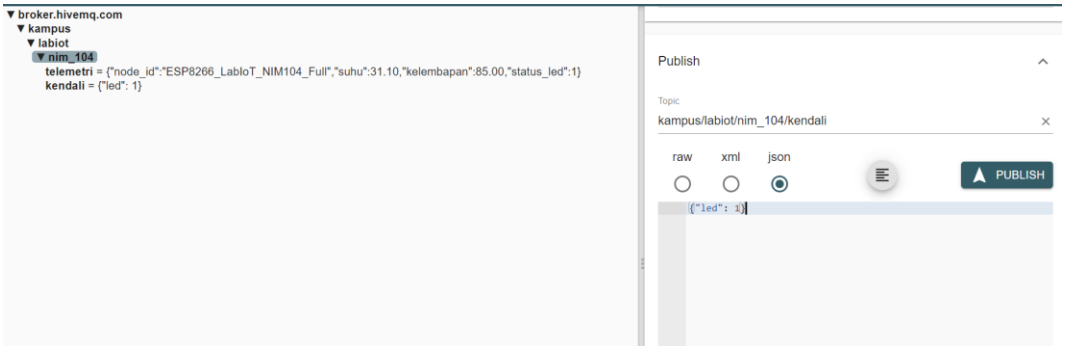
6.5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

Pada praktikum ini, kita menyatukan kapabilitas Praktikum 1 dan Praktikum 2 ke dalam sebuah sistem terpadu (*Full-Duplex Telemetry & Control*). NodeMCU secara kontinu memublikasikan data telemetri sensor lingkungan (Suhu, Kelembapan, dan Status LED fisik) ke Broker setiap 4 detik, sembari tetap responsif mendengarkan dan mengeksekusi instruksi kontrol LED dari topik perintah kapan saja.

Hasil Serial:

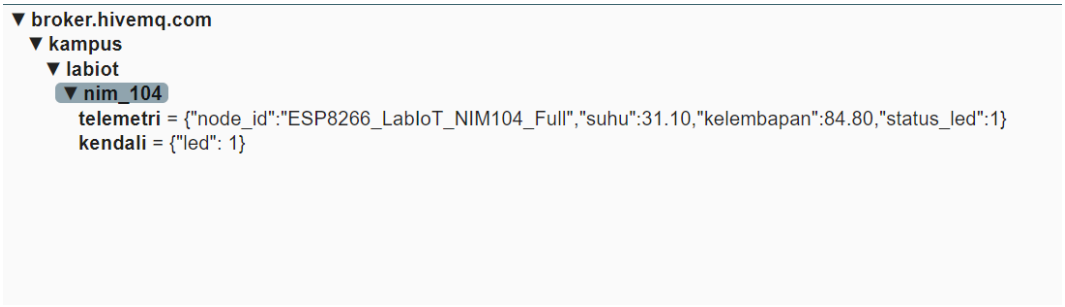
```
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":84.70,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":84.80,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":84.80,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":84.90,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":84.90,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":85.00,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":84.90,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":84.90,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":85.00,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":84.90,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.10,"kelembapan":85.10,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":85.20,"status_led":0}
----------------------------------------
[KONTROL DITERIMA] Topik: kampus/labiot/nim_104/kendali | Payload: {"led": 1}
--> [INDIKATOR] Status LED diubah: MENYALA
----------------------------------------
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":85.10,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":84.80,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":84.60,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":84.60,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_LabIoT_NIM104_Full","suhu":31.00,"kelembapan":85.70,"status_led":1}
```

Perintah di MQTT broker:

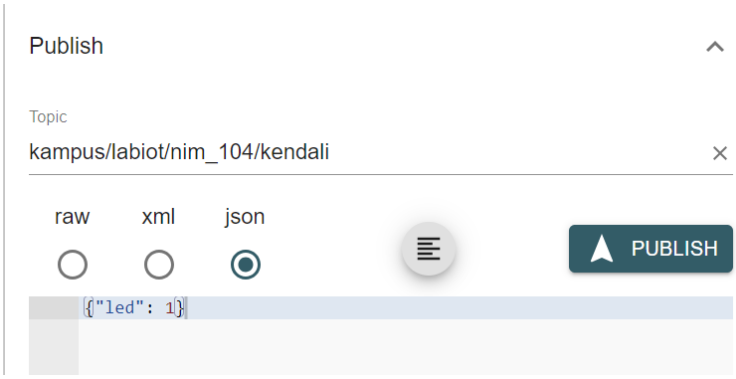


6.5.4 Panduan Pengujian Diagnostik dengan MQTT Explorer

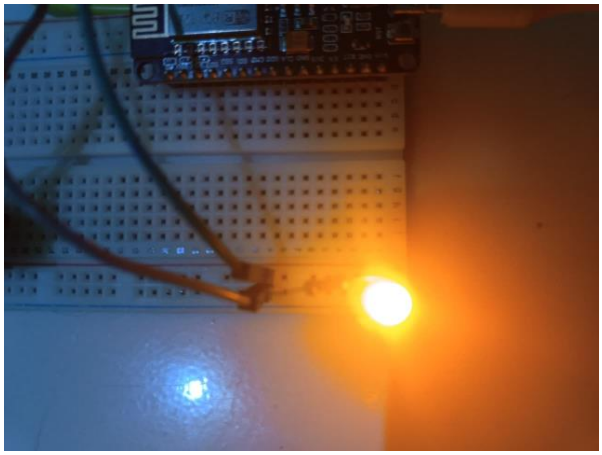
1. **Jalankan Aplikasi MQTT Explorer** pada laptop Anda.
2. **Konfigurasi Koneksi Broker:**
    - **Protocol:** mqtt://
    - **Host:** broker.hivemq.com
    - **Port:** 1883
    - Klik tombol **Connect**.
3. Memantau Data Masuk (Telemetry Monitoring):



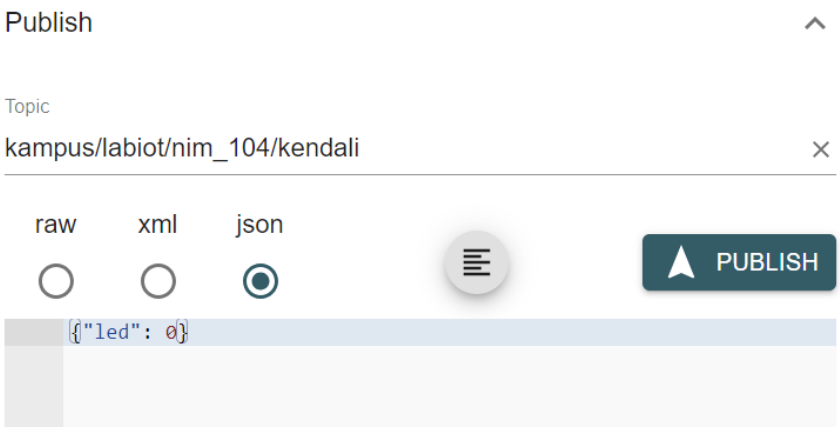
4. Menerbitkan Instruksi Kendali (*Publish Command*):



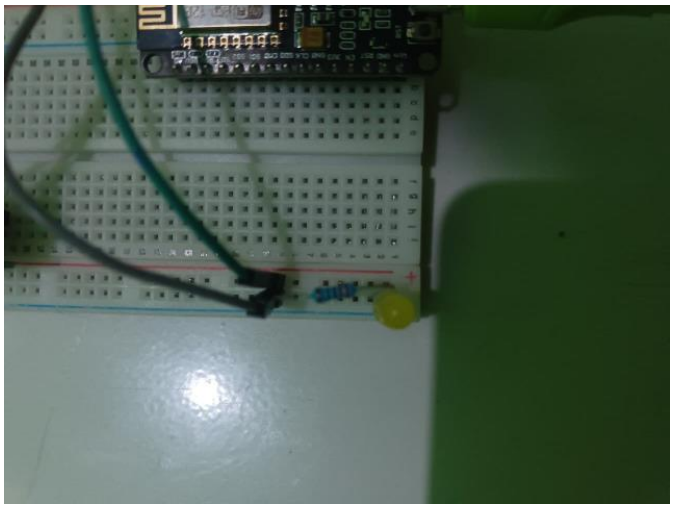
Hasil:



Perintah publish:



Hasil:



6.6 Latihan

Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):

Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:

i. Efisiensi penggunaan kapasitas *bandwidth* pada paket data seluler.

MQTT: Menggunakan struktur header yang sangat ringkas, yaitu berkisar antara 2 hingga 5 byte per paket data. Protokol ini mempertahankan koneksi tunggal yang persisten (persistent TCP connection) berbasis keep-alive, sehingga tidak ada overhead handshake berulang.

HTTP: Merupakan protokol stateless yang mengharuskan pengiriman header HTTP lengkap pada setiap permintaan (request) dan respons (response). Ukuran header HTTP bisa mencapai beberapa ratus byte hingga 1 kilobyte (berisi informasi User-Agent, Accept, Connection, Content-Type, dll.), yang membuang bandwidth seluler secara signifikan untuk data sensor berukuran kecil.

ii. Beban komputasi mikrokontroler (*CPU overhead*) saat menangani pertukaran data dua arah.

MQTT: Pertukaran data dua arah (*bidirectional*) terjadi secara *push-based* melalui koneksi TCP yang terus terbuka. Mikrokontroler hanya perlu mendengarkan *socket* (diwakili fungsi client.loop()), sehingga beban CPU sangat kecil dan hemat energi.

iii. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (*1-to-N distribution*).

MQTT: Menggunakan pola arsitektur Publish/Subscribe berbasis Broker. Ketika satu perangkat (*Publisher*) memublikasikan data ke suatu topik, Broker MQTT bertugas merutekan dan mendistribusikan data tersebut secara simultan ke ratusan atau ribuan *Subscriber* sekaligus tanpa menambah beban komputasi maupun bandwidth dari pengirim data awal.

Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):

Sebuah pabrik manufaktur memiliki 3 lini produksi (*line1*, *line2*, *line3*), di mana setiap lini memiliki 4 mesin (*m1*, *m2*, *m3*, *m4*), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik/<line>/<mesin>/<sensor>. Tentukan ekspresi topik subskripsi (*subscription topic*) yang tepat menggunakan karakter *wildcard* (+ atau #) untuk:

i. Memantau parameter suhu dari seluruh mesin di semua lini produksi.

Ekspresi Topik: pabrik/+/+/suhu

Penjelasan: Wildcard single-level + digunakan pada tingkat <line> dan <mesin> untuk mencocokkan baris lini dan mesin mana saja.

ii. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2.

Ekspresi Topik: pabrik/line2/m1/# (atau pabrik/line2/m1/+)

Penjelasan: Wildcard multi-level # (atau single-level +) di tingkat akhir mencocokkan seluruh jenis sensor (suhu, getaran, status) pada line2 dan mesin m1.

iii. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut

Ekspresi Topik: pabrik/#

Penjelasan: Wildcard multi-level # mencocokkan semua sub-topik di bawah root topic

Latihan 3 (Implementasi Serialisasi Data Dinamis):

Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

i. "uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).

ii. "ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()).

Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!

```
doc["uptime_detik"] = millis() / 1000;
doc["ip_lokal"] = WiFi.localIP().toString();
```

```
........
Wi-Fi Terkoneksi!
Alamat IP Lokal: 192.168.1.52
Menghubungkan ke MQTT Broker... Sukses Terhubung!
Telemetri Terkirim [kampus/labiot/nim_104/telemetri]: {"node_id":"ESP8266_LabIoT_NIM104","suhu":30.20,"kelembapan":87.40,"uptime_detik":5,"ip_lokal":"192.168.1.52"}
Telemetri Terkirim [kampus/labiot/nim_104/telemetri]: {"node_id":"ESP8266_LabIoT_NIM104","suhu":30.20,"kelembapan":87.40,"uptime_detik":10,"ip_lokal":"192.168.1.52"}
Telemetri Terkirim [kampus/labiot/nim_104/telemetri]: {"node_id":"ESP8266_LabIoT_NIM104","suhu":30.20,"kelembapan":87.30,"uptime_detik":15,"ip_lokal":"192.168.1.52"}
```

Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):

Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().

i. Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah *callback* jika seorang pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop()?

Fungsi client.loop() berfungsi untuk memproses paket data MQTT yang masuk dari broker, memelihara sinyal keep-alive (ping), dan mengeksekusi fungsi callback.
Jika diberikan delay(6000) (blocking selama 6 detik), mikrokontroler akan menghentikan seluruh eksekusi program selama rentang waktu tersebut. Akibatnya:
Responsivitas Callback Tertunda: Perintah kontrol dari broker yang dikirimkan saat delay tidak akan langsung dieksekusi, melainkan tertahan hingga client.loop() dipanggil kembali.

Potensi Diskoneksi (Keep-Alive Timeout): Jika waktu delay melebihi interval keep-alive MQTT, broker akan menganggap NodeMCU terputus (offline) dan memutus paksa koneksi socket TCP (ping timeout)

ii. Jelaskan bagaimana fungsi reconnect() bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!

Fungsi reconnect() menggunakan perulangan while (!client.connected()) untuk secara kontinu memeriksa status koneksi ke broker MQTT.
Jika terputus: Fungsi akan memanggil client.connect(clientId).
Saat koneksi berhasil dipulihkan: Mikrokontroler akan segera memanggil ulang fungsi client.subscribe(topicSubscribe). Langkah ini penting karena setelah koneksi TCP terputus dan tersambung kembali, sesi pendaftaran topik di broker publik hilang (jika *clean session* bernilai true), sehingga subskripsi topik perintah harus didaftarkan ulang agar *callback* tetap dapat menerima pesan secara normal.

6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri

Klasifikasi Status Lingkungan pada Telemetri Rutin:

```
// Ambang Batas Suhu Kritis (Edge Safety Engine)
const float TEMP_THRESHOLD = 31.0;

// Penambahan field "kondisi_suhu"
docOut["kondisi_suhu"] = (t > TEMP_THRESHOLD) ? "PANAS" : "NORMAL";
```

Kanal Peringatan Bahaya Mandiri (*Dedicated Alert Topic*):

```
// LOGIKA CLOSED-LOOP SAFETY & DEDICATED ALERT TOPIC +++
if (t > TEMP_THRESHOLD) {
  // Otomatis nyalakan LED saat bahaya overheat
  digitalWrite(ledPin, HIGH);

  // Kirim alert hanya saat pertama kali bahaya terdeteksi (State Latchin
  if (!isOverheatAlertActive) {
    isOverheatAlertActive = true;

    StaticJsonDocument<256> docAlert;
    docAlert["status"]     = "BAHAYA";
    docAlert["parameter"]  = "OVERHEAT";
    docAlert["nilai_suhu"] = serialized(String(t, 1));

    char alertBuffer[256];
    serializeJson(docAlert, alertBuffer);
    client.publish(topicAlert, alertBuffer);
    Serial.printf("\n[ALERT BAHAYA TERKIRIM] %s\n\n", alertBuffer);
  }
} else {
  // Reset latch jika suhu kembali normal
  isOverheatAlertActive = false;
}
```

Mekanisme Last Will and Testament (LWT):

```
// Payload Last Will and Testament (LWT) +++
const char* lwtPayload = "{\"status\":\"OFFLINE\",\"keterangan\":\"Perangkat Terputus Tak Terduga\"}";

// Connect dengan 7 parameter: ClientID, User, Pass, WillTopic, WillQoS(1), WillRetain(true), WillMessage
if (client.connect(clientId, NULL, NULL, topicStatus, 1, true, lwtPayload)) {
  Serial.println(" Sukses Terhubung!");

  // Publikasi pesan ONLINE (Retain: true) saat berhasil terhubung +++
  const char* onlinePayload = "{\"status\":\"ONLINE\",\"keterangan\":\"Perangkat Beroperasi Normal\"}";
  client.publish(topicStatus, onlinePayload, true);
  Serial.printf("[STATUS LWT] Status ONLINE dikirim ke: %s\n", topicStatus);
```

Kendali Manual Override Saat Normal:

```
void callback(char* topic, byte* payload, unsigned int length) {
  char incomingBuffer[length + 1];
  memcpy(incomingBuffer, payload, length);
  incomingBuffer[length] = '\0';

  Serial.println("----------------------------------------");
  Serial.printf("[KONTROL DITERIMA] Topik: %s | Payload: %s\n", topic, incomingBuffer);

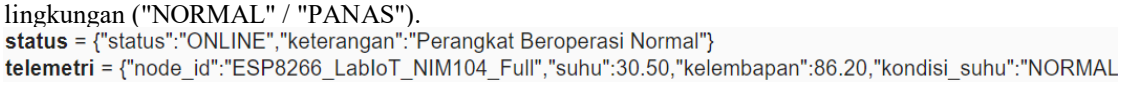
  StaticJsonDocument<256> doc;
  DeserializationError err = deserializeJson(doc, incomingBuffer);
  if (err) {
    Serial.println("Gagal membedah payload JSON!");
    return;
  }

  // Menerima parameter "led" maupun "status_led"
  int val = -1;
  if (doc.containsKey("led")) {
    val = doc["led"];
  } else if (doc.containsKey("status_led")) {
    val = doc["status_led"];
  }

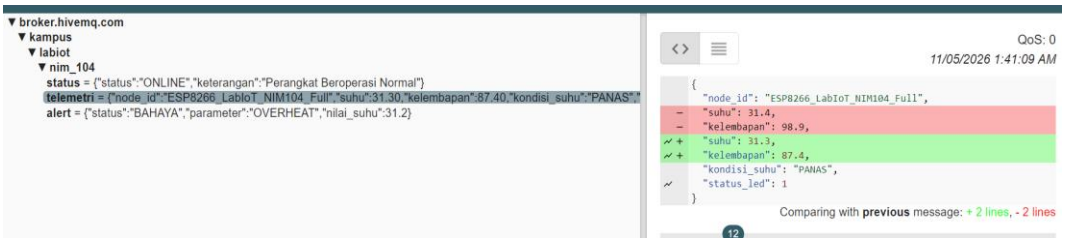
  if (val == 1 || val == 0) {
    digitalWrite(ledPin, val == 1 ? HIGH : LOW);
    Serial.printf("--> [INDIKATOR] Status LED diubah: %s\n", val == 1 ? "MENYALA" : "PADAM
  }
```

Hasil:

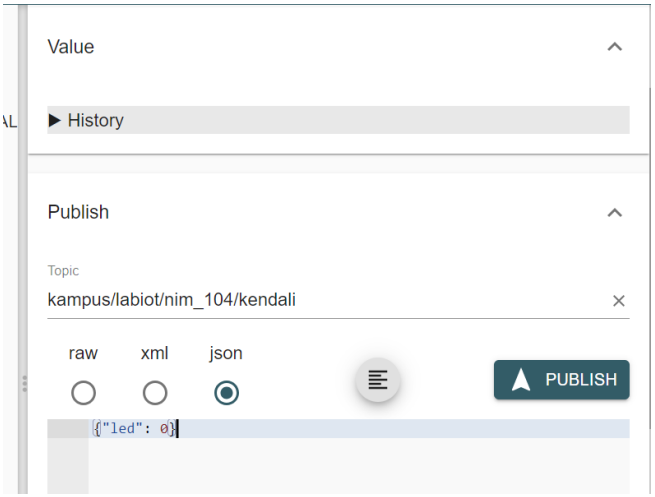
1. Data telemetri periodik terkirim stabil setiap siklus waktu dengan field klasifikasi kondisi lingkungan ("NORMAL" / "PANAS").



2. Saat sensor DHT dipanaskan hingga melebihi ambang batas 31.0∘C, NodeMCU memublikasikan data peringatan ke topik alert serta otomatis menyalakan LED indikator bahaya.



3. Operator luar tetap dapat mengambil alih kendali manual (*override*) LED melalui topik kendali saat kondisi suhu aman.



4. Mekanisme LWT terbukti bekerja saat kabel daya mikrokontroler dicabut mendadak.

